NON- PRIMITIVE TYPE CASTING

The process of converting Non- primitive type to another non-primitive type, we call it as Non-primitive type casting.

Non- primitive type casting is not possible neither implicitly nor explicitly for the classes which doesn't have any relationship b/w them.

Eg:

```
class A
{

}
class B
{
}
class Driver1
{
 public static void main (String [] args)
  {
     A obj = new A();

     B obj= new B();

     B obj1= obj; // CTE implicit conversion not possible.
   }
}
```

02/2022

Eg: °class A
{
}
class B
{
}
class Driver2
{
public static void main (String [] args)
{
A obj = new A();
B obj1 = (B) obj; //CTE
}
}

which

NOTE: We can convert one Non-primitive type to another Non-primitive type only when we have a Is-A relationship between two classes.

The Non-primitive type casting is possible only in the below situations:

1) Whenever we have Is-A relationship between two classes.

2) Whenever we have a common child [Multiple Inheritance].

Eg:

```
class A
{
  int i = 10;
}
class B extends A
{
  int j = 20;
}
class Driver1
{
  public static void main (String [] args)
  {
    B obj = new B();
    System.out.println (obj.j);
    System.out.println (obj.i);
    A obj1 = obj;
    System.out.println (obj);
    System.out.println (obj1);
    System.out.println (obj1.i);
    System.out.println (obj1.j); //CTE
  }
}
```

In the above example we are converting one reference type (non-primitive type) to another reference (non-primitive type) bcz we have IS-A relationship between both the classes.

The Non-primitive type casting can be classified into two types:

1) Up casting
2) Down casting

UP-CASTING:

The process of converting sub-class reference type into super-class reference type is known as Up-casting. [child to parent conversion]:

Upcasting will be done implicitly. Hence, it is also known as Auto up casting.

Eg:
```
class A
{
  int i=10;
}
class B
{
  int j=20;
}
class Driver2
{
  public static void main(String[] args)
  {
      B obj = new B();
      A obj1 = obj; //CTS → upcasting - done implicitly
      A obj2 = new B(); //CTS → upcasting - done implicitly
  }
}
```

SIDE EFFECT / DRAWBACK / DISADVANTAGE OF UPCASTING:

1. Whenever we up cast one reference type to its Parent type, we will not be able to access sub-class members with the help of superclass

reference.

Eg:
```
class B extends A
{
 int j= 20;
}
class A
{
 int i= 10;
}
class Driver1
{
  public static void main (String [] args)
  {
     B obj= new B();
     System.out.println(obj.j);
     System.out.println(obj.i);
     A obj1 = obj;
     System.out.println(obj);
     System.out.println(obj1);
     System.out.println(obj1.i);
     System.out.println(obj1.j); //CTE
  }
}
```

NOTE: System.out.println(obj1.j) - In the above pgm we will have compile time error because of the instruction written above, which can be explained using any one of the following ways;

1) With the help of Semantics :-

obj1.j

reference.

Eg:
```
class B extends A
{
 int j= 20;
}
class A
{
 int i= 10;
}
class Driver1
{
  public static void main (String [] args)
  {
    B obj= new B();
    System.out.println(obj.j);
    System.out.println(obj.i);
    A obj1 = obj;
    System.out.println(obj);
    System.out.println(obj1);
    System.out.println(obj1.i);
    System.out.println(obj1.j); //CTE
  }
}
```

NOTE: System.out.println(obj1.j) - In the above pgm we will have compile time error because of the instruction written above, which can be explained using any one of the following ways;

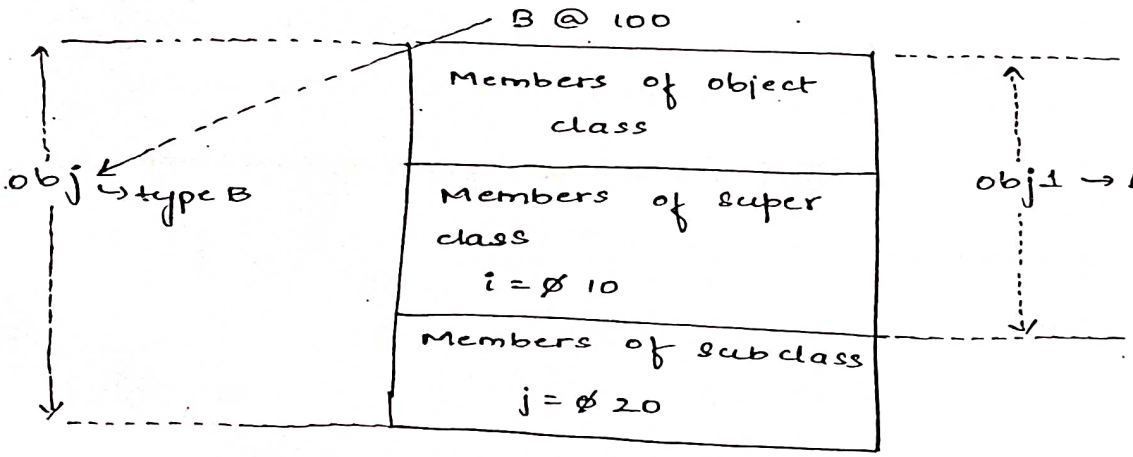
1) With the help of Semantics:-

obj1.j

During the semantics checking of above example, the compiler will check whether the variable j is present inside obj1 type ie class A. Inside class A we dont have variable j neither declared nor Inherited. Hence we get Compile Time Error.

2) With the help of visibility of super class Variables :

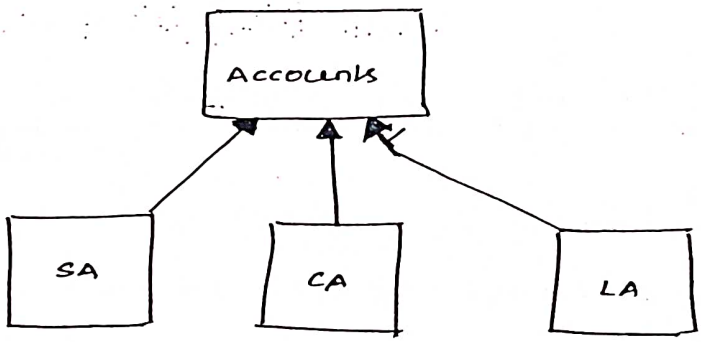
Whenever we have reference variables created for a class which is involved in Is-A relationship with other class, that reference variable will have the visibility only to its members of its class as well as its super class. It will not be able to see the members of its sub class.



▶ ADVANTAGE OF UPCASTING:

⇒ Upcasting helps us to create a generalised container inside which we can store any of the sub class references

Eg: let us consider the Accounts heirarchy as shown below:-



let us Assume a bank object which will have Accounts inside it. ie Bank object and account object will have has a relationship. Therefore inside the class bank we need to create a container called as a1 which of type accounts.

Eg:
```
class Bank
  {
    Accounts a1; // having reference variable
  }
class Accounts
{
}
```

```
class SA extends Accounts
{
  String name;
  SA(){ }
  SA (String name)
  {
   this.name= name;
    }
  }
class CA extends Accounts
{
}
class LA extends Accounts
  {
  }
  class Driver3
  {
   public static void main (String[] args)
   {
    Bank obj= new Bank();
    System.out.println (obj.a1);  //null
    obj.a1 = new SA("sheela");
    System.out println (obj.a1); //SA@101
    obj.a1= new CA();
    System.out.println (obj.a1); //CA@102
    obj.a1= new LA();
    System.out.println (obj.a1); //LA@103.
   }
  }
```

NOTE : To overcome the problem of upcasting we will go for Down-casting

▶ DOWN-CASTING :

The process of converting super class reference type into sub class reference type is known as Down-Casting.

Downcasting will never happen implicitely, we will get CTE

Eg:
```
class A
{
 int i=10;
}
class B extends A
{
 int j=20;
}
class Driver1
{
 public static void main(String[] args)
  {
    A obj = new B(); //CTS → upcasting
    B obj1 = obj; //CTE → Downcasting
  }
}
```

NOTE : Down casting must be done explicitely with the type cast operator.

Eg:
```
class A
{
 int i=10;
}
class B
{
 int j=20;
}
class Driver2
{
  public static void main(String[] args)
  {
    A obj = new B();
    B obj1 = (B) obj; //CTS.- Down casting done explicitly
  }
}
```

▶ ADVANTAGES:

1) The main advantage of Down casting is we will be able to access the members of subclass.

Eg:
```
class A
{
 int i=10;
}
class B
{
 int j=20;
}
class Driver3
{
 public static void main(String[] args)
 {
   A obj = new B();
   //System.out.println(obj.j) //CTE bcz cannot access sub class member frm superclass reference
   B obj1 = (B) obj;
   System.out.println(obj1.j);
   System.out.println(obj1.i);  // 1.
```

DRAWBACKS:

ClassCastException :

It is a run time problem which occcurs during down casting, if the created object ~~doesn't bare the members~~ doesn't bare the members of sub-class.

NOTE : 1) To get ClassCastException, we must create Super class object

2) We bave to downcast the superclass reference to sub-class reference.

Eg:
```
class A
{
  int i=10;
}
class B extends A
{
  int j=20;
}
class Driver4
{
  ~~Sys~~ public static void main (String[] args)
  {
    System.out.println("Main Begin");
    A obj = new A();
    System.out.println(1);
    B obj1 = (B) obj; //downcasting  R.t - ClassCastException
    System.out.println(2);
    System.out.println(obj1.j);
  }
}
```

NOTE:

We can avoid ClassCastException by checking whether the Object has the instance of a particular class before down casting it. We can do this with the help of an Operator known as "instanceof" operator.

▶ instanceof :

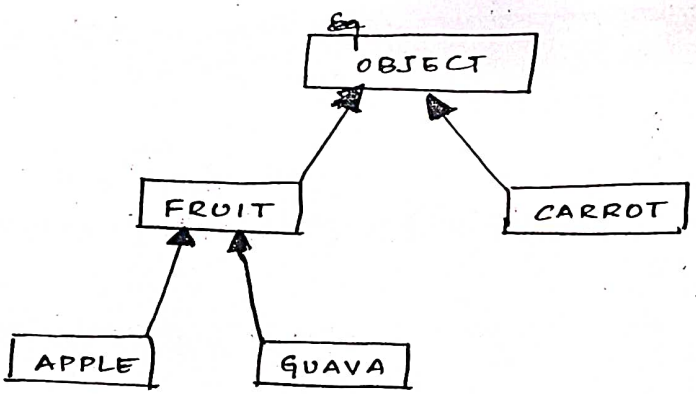
- It is a keyword.
- It is a binary operator.
- It is used to check whether an Object has instance (members) of a particular class or not.
- The return type of instanceof operator is "boolean".

NOTE : We can check the instance of a particular class if the created object reference type & the class to which we are checking is having "Is-A relationship". Else, we get CTE.

SYNTAX :

```
Object-reference instanceof ClassName ;
```

Eg:




```
class Object
{
}
class Fruit extends Object
{
}
class Carrot extends Object
{
}
class Apple extends Fruit
{
}
class Guava extends Fruit
{
}
class Driver4
{
 public static void main (String [] args)
 {
      Fruit obj= new Apple();
      System.out.println (obj instanceof Object); // true
      System.out.println (obj instanceof Fruit); //true
      System.out.println (obj instanceof Apple); //true
      System.out.println (obj instanceof Guava); // false
      System.out.println (obj instanceof Carrot); //CTE
 }
}
```

```
class Drivers
{
  public static void main (String [] args)
  {
     Object obj = new Apple();
     S.o.pln (obj Instanceof Object); // true
     S.o.pln (obj instanceof Fruit); //true
     S.o.pln (obj instanceof Apple); //true.
     S.o.pln (obj Instanceof Guava); //false
     S.o.pln (obj instanceof Carrot); //false.
  }
}
```

